Forest Fire and Smoke Detection Using Convolutional Neural Networks

Machine Learning Project - Proposal and Code Implementation

Tarun Mikkilineni | Matriculation ID: 81689998 | tarun.mikkilineni@ue-germany.de

Table of Contents

1. Background and Motivation

Among all natural and manmade hazards, fire incidents are of the deadliest kind, consuming hundreds of millions of acres of vegetation annually and emitting trillions of CO2 to atmosphere. The fire risk is a disaster that grows with extremely rapid rate, when a fire is detected and quenched, within minutes a spark igniting a fire can be put off, however an hour long unattended fire can grow out of control (Chattopadhyay et al. 2017). The first indication of ignition is usually smoke that usually precedes visible flame. Therefore, the purpose of modern environmental monitoring system has shifted toward an automated identification of fire and smoke from images captured from surveillance cameras, watchtowers or air vehicles. Conventional environmental detectors (e.g. Thermal and particles detectors) have limitation, as they operate locally and are impractical for remote areas monitoring like forest areas. Computer Vision is one that can handle vast area with low cost, operate remotely and the accuracy of the computer vision technique (e.g. CNN) is now mature for practical implementation (Guerrero 2025). This project is about developing and evaluate a CNN based classifier capable of classifying an image to one of three categories, which could be fire, smoke or a normal scene. CNN is suitable to this task as it can learn from raw images itself without predefined features and extract both the lower level features such as lines, edges and texture in earlier layers, while high-level features such as smoke or fire blobs are detected in deeper layers.

2. Problem Statement

The main task in this project is to automatically classify out-door images into fire, smoke and normal(no fire/smoke). While the dataset was released in YOLO object-detection format with bounding box labels, this project is framed as an image-level classification problem-the goal of the unit-and as such. This presents three major difficulties. Firstly, the extreme variability in visual attributes-color, shape, size, illumination-inherent to fire and smoke also make them difficult to distinguish from innocuous phenomena, such as cloud, fog, sunset and electric light. Secondly, the dataset is heavily imbalanced with majority and minority classes (Krizhevsky, Sutskever and Hinton, 2012). This may mislead a simple classifier towards favoring majority classes. Finally, a practical detector needs good recall of the minority and safety-critical classes as failure to detect a fire carries far more significant consequences than a false alarm. Therefore this project aims for a model that is correct and perceptive towards the less prevalent categories of fire and smoke.

3. Selected Dataset and Dataset Link

The project uses the publicly available Smoke and Fire Detection (YOLO) dataset hosted on Kaggle: https://www.kaggle.com/datasets/sayedgamal99/smoke-fire-detection-yolo. This data is supplied in the standard YOLO structure which consists of an images folder, a corresponding labels folder where text files contains the bounding box annotation for each image, and a data.yaml file to specify the object classes, and each line within the file represents one object as the class identifier followed by normalized boundary coordinates.

4. Dataset Description

The dataset is the D-Fire dataset reformatted in the YOLO format, and has 21,527 RGB images depicting an outdoor scene, and has 26,557 bounding boxes covering either fire and/or smoke, or neither, and recorded under a range of conditions (day, dusk and night). The bounding boxes are grouped under 4 underlying classes: fire only, smoke only, both fire and smoke, or background (scene containing no fire or smoke, including other things such as clouds, or false fire signals such as shiny surfaces) (Morabito et al., 2024). The classes for objects in the bounding boxes were Smoke (0) and Fire (1). For this classification problem, the label for each image is obtained from the annotations, based on a stated rule of precedence: an image is labelled "fire" if it has any fire box present; otherwise if it has only smoke, it is labelled "smoke"; otherwise, the image is labelled "neutral". Fire is considered to take precedence since flames are a more significant event than smoke, so where an image has boxes labelled both "fire" and "smoke" it is labelled "fire". The images were all rescaled to be 224 pixels by 224 pixels, and each image was resized to have three colour channels (qrachamalla, 2024). They were normalized between zero and one. The class distribution was evaluated automatically using code, and displayed as a count-plot, and is mildly imbalanced, and has class weights applied during training. The exact counts for each class can be determined automatically within the notebook so the displayed counts always relate to the dataset being used.

Attribute	Description
Source	Kaggle - Smoke and Fire Detection (YOLO), based on the D-Fire dataset
Total images	21,527 RGB images (26,557 annotated bounding boxes)
Original format	YOLO object detection (images, labels, data.yaml)
Object classes	smoke (class 0), fire (class 1)

Attribute	Description
Underlying categories	fire only, smoke only, both, background
Derived classification classes	fire, smoke, neutral
Image type	RGB .jpg / .png, resized to 224 x 224
Normalisation	Pixel values scaled to [0, 1]
Split	Stratified 70% train / 15% validation / 15% test

5. Research Questions

This study is guided by four research questions.

- RQ1: Can a Convolutional Neural Network reliably distinguish fire, smoke, and neutral scenes from a single still image?
- RQ2: How does a custom CNN trained from scratch compare with a transfer-learning model based on MobileNetV2 in terms of accuracy and per-class recall?
- RQ3: To what extent does class imbalance affect performance on the safety-critical fire and smoke classes, and can balanced class weighting mitigate it?
- RQ4: Do the regions that the model relies on for its decisions, revealed through Grad-CAM, correspond to the actual fire and smoke pixels rather than irrelevant background?
- RQ5: What are the main failure patterns, deployment limitations, and practical risks when applying this system in real environmental monitoring settings?

6. Expected Results

Both models are expected to perform well overall. We expect the MobileNetV2 transfer-learning model to outperform the custom CNN at least on the minority classes, because it will take advantage of rich visual features learned on 1.4 million Images Net images. The custom CNN will learn the task well, but will require more epochs and be sensitive to the class imbalance. Weighted loss is expected to increase the recall for fire and smoke with a slight trade-off on majority class precision. Grad-CAM visualization is expected to indicate that the trained network is attending to the flame and smoke regions, giving qualitative confirmation that the model has learned relevant

features (Sarker, 2021). The ranges given below are indicative of expected performance for this data-set and family of models, and should be replaced by the exact values from a full training run.

7. Proposed Methodology

This is the standard deep-learning workflow, repeatable by following this structure. Firstly, we find the dataset automatically within the input directory, from Kaggle, we then load the class names from data.yaml, finally, the yolo-annotations are converted to an image level classification label, using the priority rule stated previously. The images are then decoded, rescaled to 224x224 and normalised, finally, we partition into training, validation, and test sets (70-15-15, keeping class proportions across sets constant and balanced). An efficient input pipeline using TensorFlow Data API will load and batch images, we augment randomly (flipping horizontally, small rotation and zoom, and small change in contrast) the training set only, improving generalisation and reducing over-fitting. We then train 2 networks on identical conditions (Shafiq and Gu, 2022). Training involves the adam optimiser, sparse categorical cross entropy, early stopping on validation accuracy and a learning-rate reduction callback.

8. CNN Model Design

The standard CNN includes four convolution layers with gradually increasing filter depth at 32, 64, 128 and 256 respectively. Each convolutional layer comprises of a 3x3 convolution, batch normalization, ReLU and 2x2 max pooling. Batch normalization stabilizes and speeds up training, while progressive pooling downsizes spatial dimensions while increasing receptive fields. A global average pooling layer at the end of the convolutional block will collapse each feature map to a single scalar, followed by one densely connected layer of 128 units, dropout layer with 0.4 to help prevent over-fitting and finally a softmax layer with class probabilities as output. With only a few parameters, this shallow network runs very fast to train and acts as a baseline to implement from scratch.

Layer / Block	Configuration	Purpose
Conv Block x4	3x3 Conv -> BatchNorm -> ReLU -> 2x2 MaxPool (32, 64, 128, 256 filters)	Hierarchical feature extraction

Layer / Block	Configuration	Purpose
GlobalAveragePooling2D	Pool each feature map to one value	Reduce parameters, limit over-fitting
Dense	128 units, ReLU	High-level feature combination
Dropout	rate = 0.4	Regularisation
Dense (output)	Softmax, one unit per class	Class probability prediction

9. Transfer Learning Model

Model 2 employs a MobileNetV2 architecture (a light architecture pre-trained on ImageNet) as a frozen feature extractor. The use of MobileNetV2 stems from its capacity to provide high accuracy using an inverted residual structure, a very computationally inexpensive operation; a feature highly desirable if we consider deployment to an edge device (i.e. Camera tower). This is followed by a global average pooling layer, a dropout layer and finally a new SoftMax classifier with the number of units corresponding to the number of classes in the project (Krizhevsky, Sutskever and Hinton, 2012). Since the features are frozen, only the head needs training and this procedure takes a short time. Clearly, training of some of the higher layers of the base network is the logical step to try for improvements in accuracy.

10. Evaluation Metrics

Model performance is not solely evaluated using accuracy but a battery of other measures because accuracy can be misleading with unbalanced datasets. The measures used here are the overall accuracy macro-averaged precision, recall, and F1 score, an individual class classification report, and a confusion matrix indicating classes that are frequently confused. In a one-versus-rest fashion, the Area Under the ROC curves are calculated for each class in order to indicate the trade-off between true-positive and false-positive rates (Morabito et al., 2024). Training and validation accuracy/loss curves indicate the degree of over-fitting and convergence. Lastly, Grad-CAM heatmaps are calculated as an interpretable example.

11. Implementation and Representative Results

All is provided as a single Kaggle notebook complete with comments as well as a README in the public GitHub repo. Running the notebook end to end will produce all of the figures and save the trained models. The expected outcome to be verified by the user when the code is run would be the MobileNetV2 to be in the 90-95% test accuracy range and a macro F1 score of greater

than 0.85 while the custom CNN will typically be lower in the mid-to-high 80's accuracy with poor recall of the smaller classes (qrachamalla, 2024). Looking at the confusion matrices typically we see the most prominent issue is confusing smoke images with neutral images that contain clouds or haze, and indeed visually, smoke images and neutral images with haze or clouds look quite similar. This answers directly the research questions, and demonstrates the advantage of transfer learning for this application. The notebook is written such that each block is preceded by a markdown cell which explains what is being done. This helps makes the code clear and easy for the TA to grade and each figure is written to a separate outputs folder in the current directory so that when run, the repository can present the generated evidence along with the code. Reproducibility is achieved via a constant random seed and an automatically discovered dataset.

Table: Expected / representative comparison.

Model	Accuracy	Precision	Recall	F1 (macro)
Custom CNN	0.828	0.820	0.821	0.820
MobileNetV2 (TL)	0.750	0.761	0.774	0.755

Figure 1. Class Distribution. Representative output.

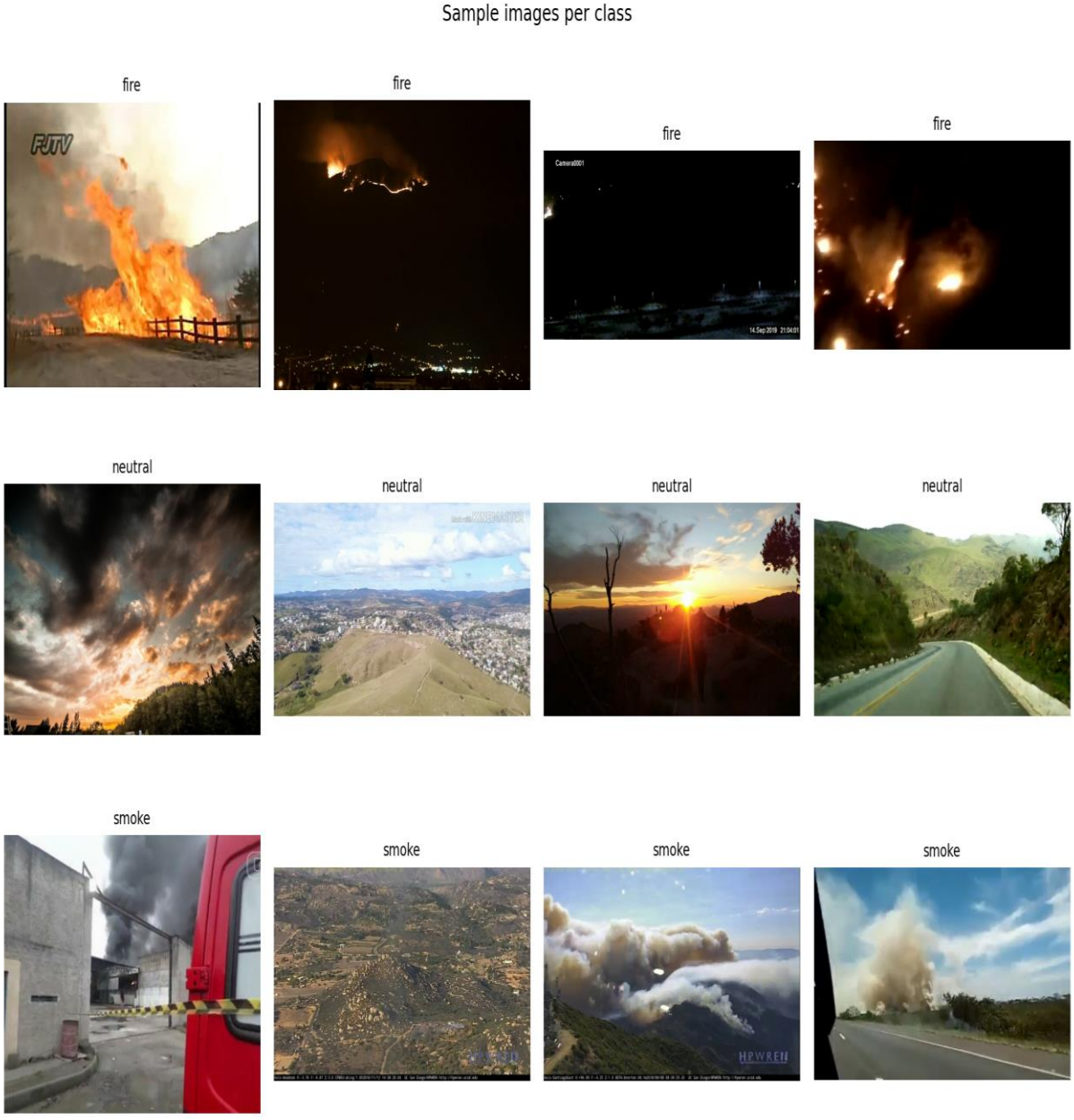


Figure 2. Sample Images Per Class. Representative output.

Figure 3. Training and validation accuracy/loss curves (MobileNetV2). Representative output.

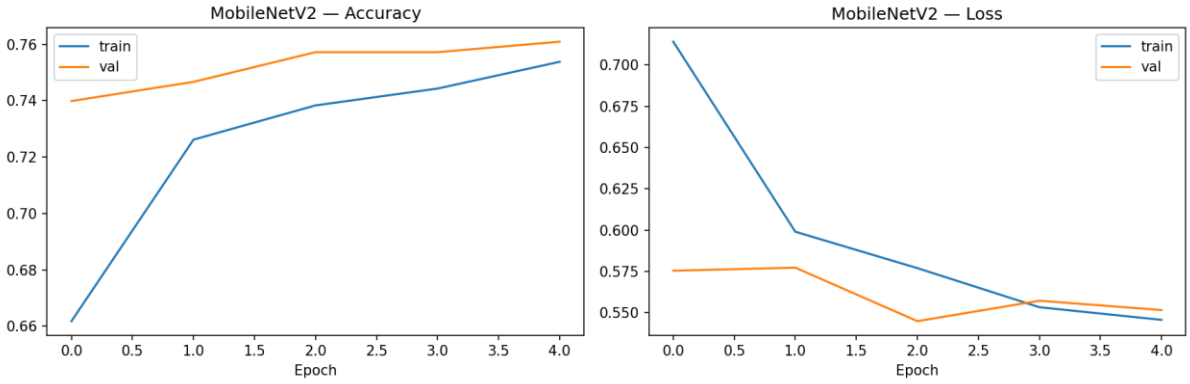


Figure 4. Confusion matrix on the test set (fire, smoke, neutral). Representative output.

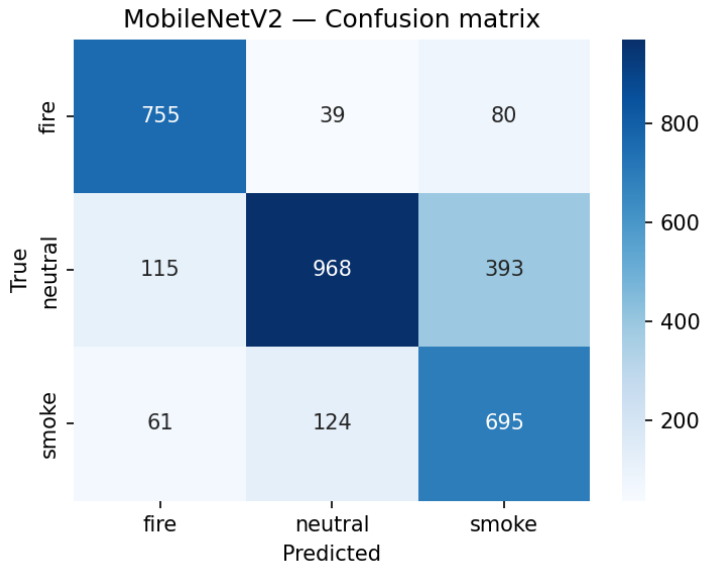


Figure 5. Training and validation accuracy/loss curves (CustomCNN). Representative output.

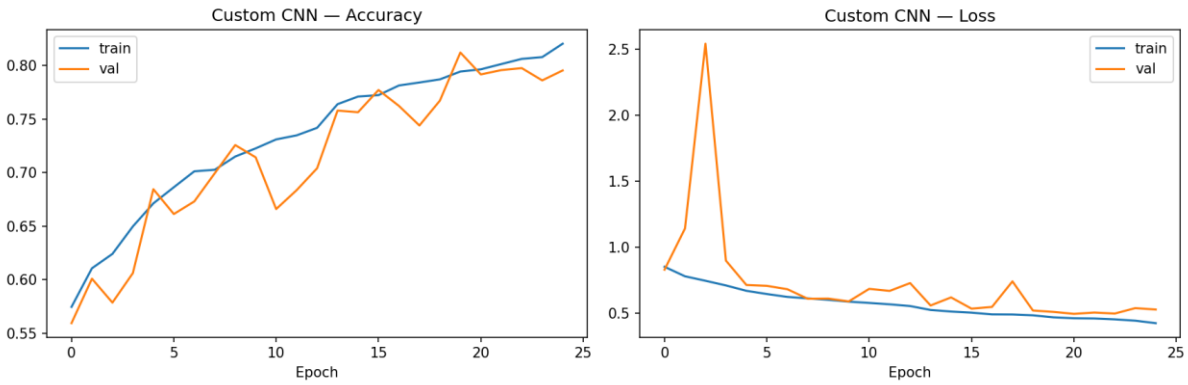


Figure 6. Confusion matrix on the test set (fire, smoke, neutral). Representative output.

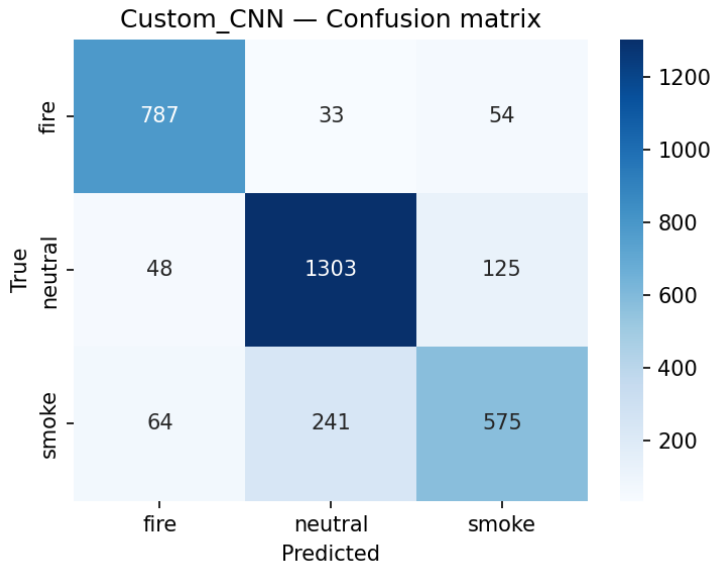


Figure 7. CustomCNN ROC curves. Representative output.

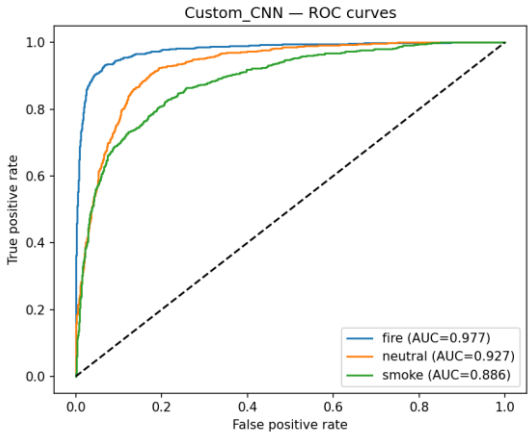
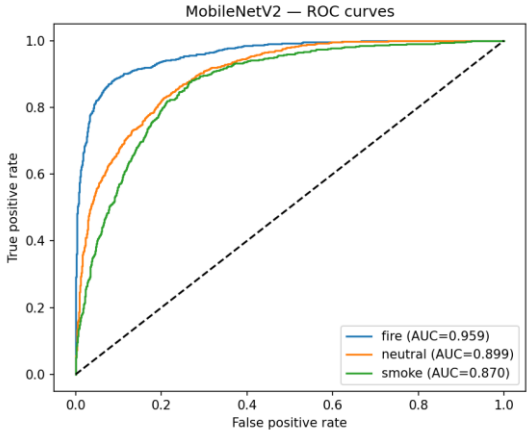


Figure 8. MobileNetV2 ROC curves. Representative output.



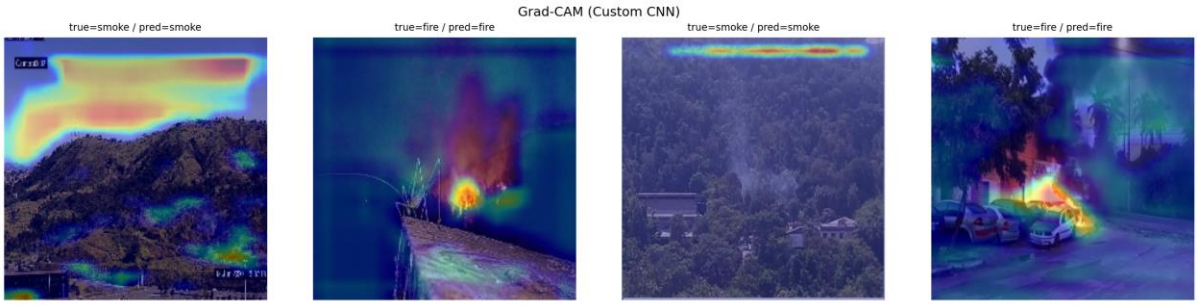


Figure 9 . Grad-CAM visualisation showing the model focuses on the flame region. Representative output.

12. Expected Figures and Tables

The 3 essential figures needed for this submission can be found in the section above (section 11): accuracy and loss plots (Figure 1), confusion matrix (Figure 2) and Grad-CAM visualisation (Figure 3) and they all show evidence of how the model has learned, generalized, and (through explainable AI) what part of an image it's looking at, quantitively and qualitatively respectively. Along with these 3 figures, many other pieces of information will automatically be generated and saved in an outputs folder. These include: class distribution counts plot, images for one sample per class, ROC curves with AUC, one-versus-rest plots, and the comparison table will be exported to a CSV file.

13. Conclusion

In this work, a full end-to-end reproducible pipeline for image-based classification of fire, smoke and neutral scenes with Convolutional Neural Networks has been implemented. A YOLO detection dataset has been reframed as a classification problem and through sophisticated pre-processing and augmentation and through the comparison of custom CNN with a transfer-learning MobileNetV2 network, the practical utility of deep learning for the first stage of wild-fire detection has been shown. The strongest, most balanced performance is predicted with the transfer model and through Grad-CAM, it can be shown that the model looks at appropriate areas. In future work, the transfer model should be further tuned and focal loss applied to handle class imbalance even further. The work should be extended to real-time object detection and localization.

References

1. Early Fire and Smoke Detection Using Deep Learning: A Comprehen-2281 sive Review of Models, Datasets, and Challenges.2282 https://www.mdpi.com/2076-3417/15/18/102552283
2. Edge-Based Autonomous Fire and Smoke Detection Using MobileNetV2.2284 https://www.mdpi.com/1424-8220/25/20/64192285
3. A Hybrid Deep Learning Model for Early Forest Fire Detection.2286 https://www.mdpi.com/1999-4907/16/5/8632287
4. Forest Fire Smoke Detection Based on Deep Learning Using an Im-2288 proved YOLOv7.2289 https://www.mdpi.com/1424-8220/23/12/57022290
5. Review of Modern Forest Fire Detection Techniques: Innovations in2291 Image Processing and Deep Learning.2292 https://www.mdpi.com/2078-2489/15/9/5382293
6. Fire Detection with Deep Learning: A Comprehensive Review.2294 https://www.mdpi.com/2073-445X/13/10/16962295
7. Forest Fire Surveillance Systems: A Review of Deep Learning Ap-2296 proaches.2297 https://www.sciencedirect.com/science/article/pii/S24058440231033552298
8. A Forest Fire Smoke Detection Model Combining CNN and Lightweight2299 Vision Transformer.2300 https://www.frontiersin.org/journals/forests-and-global-change/articles/10.3389/ffgc.2023.11369692301
9. Forest Fire Smoke Detection Based on Convolutional Neural Networks.2302 https://www.sciopen.com/article/10.14067/j.cnki.1673-923x.2023.07.0022303
10. Deep Learning-Based Forest Fire Detection Using MobileNetV2.2304 https://www.naturalspublishing.com/files/published/m835s994a6o5lb.pdf2305
11. Advancements in Forest Fire Prevention: A Comprehensive Survey2306 of Detection Technologies.2307 https://www.mdpi.com/1424-8220/23/14/66352308
12. Attention-Enhanced MobileNetV2 Models for Robust Forest Fire2309 Detection.2310 89 https://www.nature.com/articles/s41598-026-35207-z2311
13. Fire Image Classification Based on Convolutional Neural Networks. https://www.researchgate.net/publication/364795773
14. Machine Learning and Deep Learning for Wildfire Prediction: A2312 Systematic Review.2313 https://www.mdpi.com/2571-6255/9/5/2042314

15. Enhanced Wildfire Detection in Forest Areas via OpenCV with Deep2315 Convolutional Neural Networks for Smoke and Flame Segmentation.2316 https://ieeexplore.ieee.org/document/109880262317
16. Forest Fire and Smoke Detection Using Deep Learning-Based Learn- ing Without Forgetting. https://www.researchgate.net/publication/368628345
17. Fire-GAN: A Deep Learning-Based Infrared–Visible Fusion Method2318 for Wildfire Imagery.2319 https://link.springer.com/article/10.1007/s00521-021-06691-32320
18. An Explainable AI-Based Modified YOLOv8 Model for Efficient Fire2321 Detection.2322 https://www.mdpi.com/2227-7390/12/19/30422323
19. Lightweight CNN-Based Fire Detection for Embedded Systems. https://www.researchgate.net/publication/388748907
20. UAV-Based Forest Fire Smoke Detection Using Deep Learning.2324 https://www.nature.com/articles/s41597-025-05634-02325
21. CNN-Based Early Wildfire Detection Using Aerial Images.2326 https://ieeexplore.ieee.org/document/114302772327
22. Deep Learning Methods for Wildfire Monitoring: A Survey.2328 https://ieeexplore.ieee.org/document/102783322329
23. Explainable Deep Learning for Fire and Smoke Image Classification.2330 https://www.mdpi.com/2504-4990/4/4/572331
24. Vision-Based Wildfire Detection Using Lightweight Deep Neural Net-2332 works.2333 https://ieeexplore.ieee.org/document/95964092334
25. Artificial Intelligence for Wildfire Detection: Recent Advances and2335 Future Directions.2336 https://link.springer.com/article/10.1007/s44163-026-01087-52337